

TCP ECN, Delay-based CC, and Fairness

(Modern Congestion Signalling and Resource Allocation)

1. Explicit Congestion Notification (ECN)

While classic TCP infers congestion through loss, **ECN [RFC 3168]** allows routers to **explicitly signal** the onset of congestion before buffers overflow and packets are dropped.

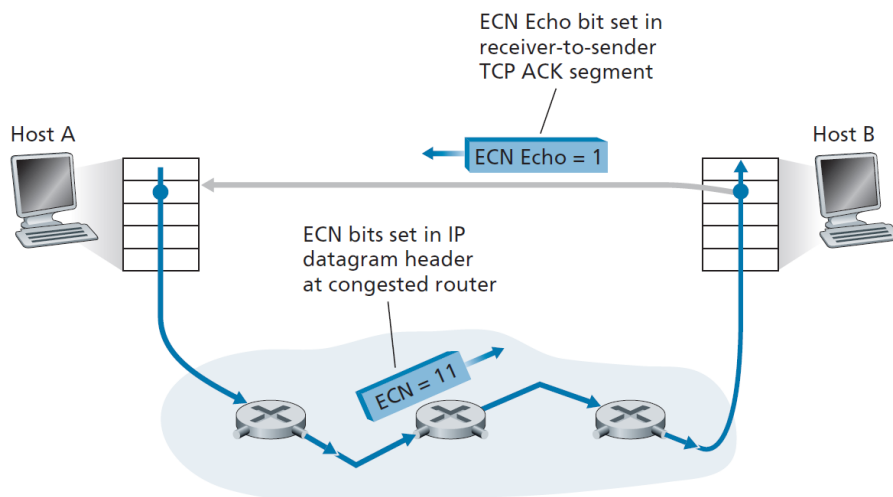


Figure 1: Explicit Congestion Notification Feedback Loop

The Marking Mechanism

1. **IP Layer:** A congested router marks two bits in the **Type of Service (ToS)** field of the IP datagram header (**ECN = 11**).
2. **Destination Notification:** The receiver sees the marked datagram and must notify the sender.
3. **TCP Feedback:** The receiver sets the **ECE (ECN Echo)** bit in its next ACK segment.
4. **Sender Reaction:** Upon receiving an ACK with **ECE=1**, the sender halves its congestion window (**cwnd**) and sets the **CWR (Congestion Window Reduced)** bit in the next data segment.

ECN-Capable Transport

The sender also uses ECN bits to inform routers that it and the receiver are **ECN-capable**. If not, routers cannot use marking and must drop packets as usual.

2. Delay-based Congestion Control

Instead of waiting for loss or explicit marks, delay-based approaches proactively detect congestion by measuring **queuing delay**.

2.1 TCP Vegas

Vegas operates on the intuition: *"Keep the pipe just full, but no fuller."*

- **Mechanism:** Measures RTT for all segments. Let RTT_{min} be the uncongested RTT.
- **Target Rate:** $Rate_{target} = cwnd / RTT_{min}$.
- **The Rule:** If actual measured throughput is close to the target, increase **cwnd**. If it is significantly lower, the path is congested, so decrease **cwnd**.

2.2 BBR (Bottleneck Bandwidth and RTT)

Developed by Google, BBR models the network's maximum bandwidth and minimum RTT. It is used extensively for YouTube and internal data center traffic as a replacement for CUBIC.

3. Fairness in TCP

A congestion-control mechanism is considered ****fair**** if K connections sharing a link of capacity R each receive an average transmission rate of R/K .

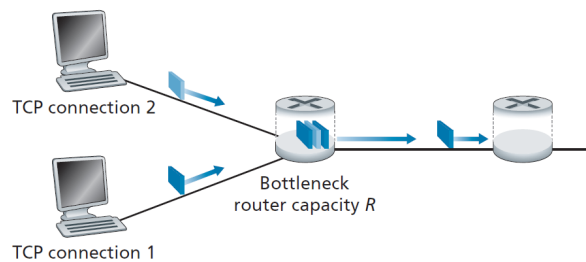


Figure 2: Two TCP Connections Sharing a Bottleneck Link

3.1 Why AIMD is Fair

The **Additive-Increase, Multiplicative-Decrease** mechanism naturally converges towards an equal share.

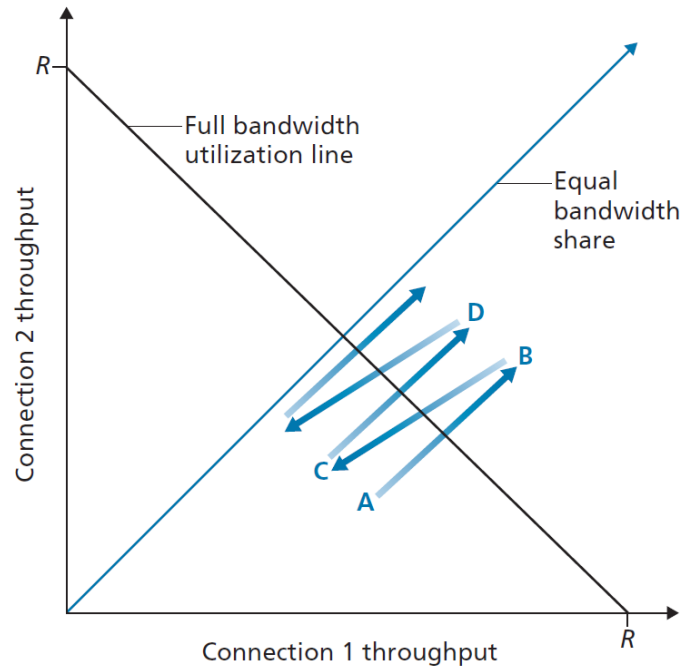


Figure 3: Throughput Convergence Vector Diagram

The Convergence Process

1. **Additive Increase:** Both connections increase throughput along a **45-degree line** (equal increase).
2. **Multiplicative Decrease:** When the sum exceeds R (full utilization line), both connections **halve** their windows, moving towards the origin.
3. **Cycle:** This process repeats, gradually shifting the "sawtooth" fluctuations closer to the **Equal Bandwidth Share** line.

4. Factors Affecting Fairness

1. Round-Trip Time (RTT)

TCP is **unfair** to connections with larger RTTs. Connections with smaller RTTs can open their windows more quickly and grab free bandwidth faster than distant connections.

2. UDP Traffic

UDP does not have built-in congestion control. In times of congestion, TCP throttles itself while UDP continues at a constant rate, potentially ****crowding out**** fair TCP traffic.

3. Parallel TCP Connections

Applications (especially web browsers) can open multiple parallel TCP connections to get a larger fraction of link bandwidth.

The Parallel Math

A link with 9 connections gives $R/10$ to a new user. If the new user opens **11 parallel connections**, they effectively grab more than $R/2$ of the link, which is clearly unfair.

5. Summary

Fairness and Extensions Recap

- **Explicit Signal:** ECN uses IP/TCP bit flags to prevent loss.
- **Delay Probing:** Vegas and BBR use RTT variations to detect early congestion.
- **AIMD Convergence:** Fairness is achieved through the geometry of window adjustment.
- **Unfairness Sources:** Different RTTs, greedy UDP, and parallel connection hacks.